

Milestone 4

LoRA Fine-Tuning for MCQ

IITM BS Data Science & Applications | Smart MCQ Solver Challenge | July 2026

1. Overview

This milestone reframes the Smart MCQ Solver task as a multiple-choice classification problem and walks through fine-tuning bert-base-uncased on it with LoRA. Where Milestone 3 used retrieval plus zero-shot classification, here the model itself learns to score each of the five options directly.

Ten questions walked through the pipeline in sequence: label encoding, prompt-option formatting, single-row and batch tokenization for the MCQ layout, a forward pass through `AutoModelForMultipleChoice` to produce logits and a scalar loss, application of LoRA to make the fine-tune tractable, dataset preparation for the Hugging Face Trainer, a tiny four-step training run, and inference with softmax to convert logits back into probabilities.

The pipeline is small on purpose. Four optimizer steps is far short of convergence, and the point is not to produce a strong classifier but to make every step concrete: what shape does the tokenized batch have, what shape do the logits have, how many parameters does LoRA leave trainable, what does the trainer's `global_step` actually count.

2. Encoding and Prompt-Option Formatting

Each training row has one prompt, five options, and one correct answer letter. The multiple-choice model expects numeric labels, so we mapped A=0, B=1, C=2, D=3, E=4 with a simple dict and stored the result in a new column. For row 150, the encoded label is 2.

The formatting convention we used for each (prompt, option) pair was:

```
str(prompt) + " [SEP] " + str(option)
```

The [SEP] token is BERT's segment separator. Using the literal string inside the prompt does not tokenize to the special-token id automatically. The tokenizer sees the seven characters "[SEP] " and splits them like any other bracketed text. In practice this does not matter much for LoRA fine-tuning because the model learns whatever segmentation cue is present. What matters is consistency: the same separator string is used at train time and at inference time.

For row 0, the formatted string built with option B has length 407 characters.

3. Tokenization for Multiple-Choice Models

`AutoModelForMultipleChoice` expects inputs shaped:

```
batch_size x num_choices x sequence_length
```

For a single row with five options and `max_length` 128, that gives [1, 5, 128]. The tokenizer, however, produces a flat batch: if we tokenize five strings at once, the output has shape [5, 128]. The extra choices dimension is a reshape after tokenization, not something the tokenizer knows about.

The reshape logic is straightforward. Tokenize $N \times 5$ strings together to get $[N \times 5, 128]$, then `view()` to $[N, 5, 128]$. This is the pattern used for both single rows (Q3) and batches (Q4). For the first 16 rows tokenized to `max_length` 128, the resulting `input_ids` tensor holds $16 \times 5 \times 128 = 10,240$ token positions.

The reason for this layout is how `BertForMultipleChoice` computes its scores. Internally it flattens the choices back into the batch dimension, passes them through BERT, then reshapes the CLS-token scores back into a `[batch, num_choices]` tensor before applying cross-entropy. The choices dimension is a bookkeeping axis for the outer forward, not something BERT itself sees.

4. Multiple-Choice Model Outputs

Once tokenized to [1, 5, 128], row 0 passed through `AutoModelForMultipleChoice` produces a logits tensor of shape [1, 5]. Five logits, one per option. Softmax over the second dimension gives a probability distribution across A, B, C, D, E.

When we also supply the correct label as a [1]-shaped long tensor, the model returns a scalar cross-entropy loss. Scalar here means a 0-dimensional tensor, not a 1-D tensor of length 1. This is the same convention used everywhere in PyTorch: reductions collapse the reduced axis entirely.

The classifier head is small. `bert-base-uncased` has no MC classifier in its pretrained checkpoint, so `from_pretrained` reports `classifier.weight` and `classifier.bias` as `MISSING` and initializes them randomly. Every logit for row 0 before training is a product of BERT's frozen features and a fresh random projection.

5. LoRA for Efficient Fine-Tuning

Full fine-tuning of a 110M-parameter model just to score five options per row is wasteful. LoRA (Low-Rank Adaptation) freezes the base model and injects small low-rank update matrices into selected layers. Only these adapters, plus the classifier head, are trained.

We used the config the milestone specified:

- `r = 8`
- `lora_alpha = 16`
- `target_modules = ["query", "value"]`
- `lora_dropout = 0.1`
- `bias = "none"`
- `task_type = TaskType.SEQ_CLS`

The trainable parameter count breaks down as:

- 12 BERT layers x 2 target modules (query and value) x 8 x (768 + 768) LoRA A and B parameters = 294,912
- Classifier head, exposed as trainable via `task_type=SEQ_CLS` `modules_to_save` (a `Linear(768, 1)` with bias) = 769
- Total: 295,681

That is 0.27% of the base model's parameters. The full model has 109,778,690 parameters; LoRA moves the training compute from all of them to 0.27% of them.

The `task_type=SEQ_CLS` choice matters more than it looks. PEFT dispatches to `PeftModelForSequenceClassification` when this is set, which auto-appends "classifier" and "score" to `modules_to_save`. Without that, the randomly-initialized classifier head would be frozen along with BERT, and the model would have no way to learn anything useful. This is the difference between a working pipeline and one that silently trains only the LoRA adapters over a frozen random classifier.

6. Hugging Face Trainer Pipeline

Before running the Trainer, the tokenized inputs, attention masks, and labels have to be packaged as a Hugging Face Dataset. Each item stores `input_ids` and `attention_mask` with shape [5, 128] (choices x sequence length) plus a scalar `labels` field. Q8 verified this: the first item's `input_ids` has five tokenized choices.

For the tiny fine-tune we used:

- 32 rows from `train.csv`
- `max_length = 64` (shorter than the 128 used for the shape-checking questions)
- `per_device_train_batch_size = 4`
- `gradient_accumulation_steps = 1`

- `max_steps = 4`

With batch size 4 and 32 examples, one epoch is 8 optimizer steps. Setting `max_steps = 4` stops training after 4 steps, which is half an epoch. The Trainer respects this override and reports `global_step = 4` in the returned `TrainOutput`.

`max_steps` overrides `num_train_epochs`. The default epochs value is 3, so without `max_steps` the model would run 24 optimizer steps. The intent here is not to train a good model. It is to demonstrate that the pipeline runs end to end, including the backward pass and optimizer step, on a real GPU with LoRA-wrapped weights.

7. Post-Training Inference and Softmax

After the four training steps, we ran the fine-tuned LoRA model on row 0 with the same [SEP]-based formatting, `max_length = 64`, and applied softmax to the resulting logits. The probability distribution across the five options was approximately:

A: 0.2033 B: 0.2007 C: 0.2028 D: 0.1965 E: 0.1967

All five probabilities cluster near 0.20 — a uniform distribution over five classes is exactly 0.20 per class. The tiny fine-tune has not moved the model far enough from its randomly-initialized classifier head to produce any real preference. The probability assigned to option E, rounded to four decimal places, is 0.1967.

This near-uniform distribution is the point of the exercise. It confirms two things:

- The pipeline is correctly wired: logits, softmax, and the choice ordering all line up.
- Real fine-tuning takes far more steps than four. A production run for this task would use hundreds or thousands of steps with a proper learning rate schedule and evaluation on a held-out split.

8. Key Findings and Observations

The [SEP] literal is a string, not a special token. When we write "[SEP]" inside the prompt-option format, the tokenizer sees seven characters, not the special separator token id 102. The distinction rarely matters for LoRA because the model adapts to whatever cue is present, but it is worth knowing. To get the real [SEP] token, we would use the tokenizer's two-sequence API (passing prompt and option as separate positional arguments) and let it insert [SEP] automatically.

The choices dimension is a reshape, not a model axis. Everything upstream of `BertForMultipleChoice`'s forward treats `[N, 5, seq]` as `[N*5, seq]`. The choices axis exists so the outer forward can reshape logits back into `[N, 5]` for the cross-entropy loss. This is why the tokenizer, which knows nothing about MCQ, produces the flat `[N*5, seq]` tensor, and we reshape it ourselves.

LoRA saves compute, not correctness. The 0.27% trainable ratio is the interesting number, but it does not mean the model learns 0.27% of what a full fine-tune would learn. It means the low-rank adapters can represent enough of the useful update to close most of the gap. On tasks where the pretrained features are already close to what is needed, LoRA works well. On tasks that require large representation shifts, it can fall short.

Four steps is not a fine-tune. The near-uniform softmax output (all probabilities near 0.20) is the diagnostic. If it had spiked hard on one option after four steps, that would be a red flag: either the pipeline is misconfigured, or the model has memorized something it should not have from four batches. Uniform output at this stage is what we expect.

The classifier is randomly initialized every load. `BertForMultipleChoice`'s classifier layer is not in the `bert-base-uncased` checkpoint. Every call to `from_pretrained` initializes it fresh. Without a seed set before `from_pretrained` runs, the output of the tiny fine-tune varies from run to run. Setting `torch` and `transformers` seeds up front is what makes the pipeline reproducible; even then, small numerical differences appear across GPUs.

Why task_type=SEQ_CLS works on a MultipleChoice model. PEFT's SEQ_CLS branch does not care that the underlying model is BertForMultipleChoice rather than BertForSequenceClassification. It only looks for a module named "classifier" or "score" and marks it as trainable. Both classes expose their final projection under the name "classifier", so the same task_type wires up the same trainable layer. Using a different task_type would leave the classifier frozen and defeat the training.

9. Libraries and Tools Used

The pipeline runs in Python on a Kaggle GPU notebook. Key dependencies:

transformers (Hugging Face). AutoTokenizer for BERT tokenization, AutoModelForMultipleChoice for the classification head, Trainer and TrainingArguments for the training loop.

peft. LoraConfig for the adapter configuration, get_peft_model to wrap the base model, TaskType.SEQ_CLS to route through PeftModelForSequenceClassification so the classifier head is trainable.

datasets. Dataset.from_list to package the tokenized rows into a form that Trainer accepts.

torch. Tensor operations, softmax, and the autograd engine that the LoRA low-rank matrices train through.

pandas / numpy. CSV loading, label encoding, and row indexing.